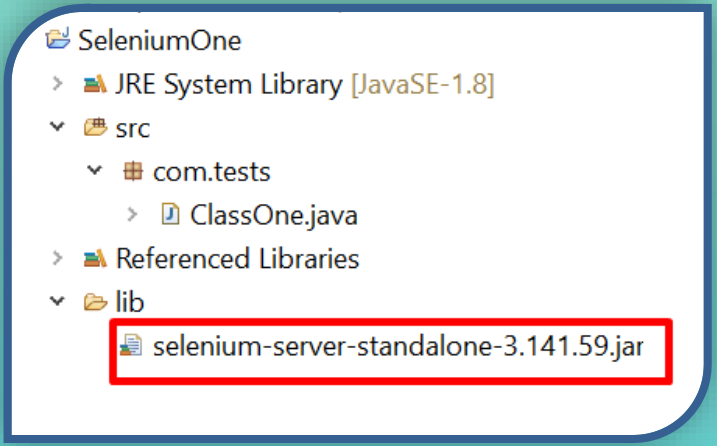
Starting With WebDriver Script
WebDirver-Lecture-2

Agenda

- Selenium Project in Eclipse
  - Download And Place Selenium Jar
  - Include Jar in Build Path
- Browser Version And Browser Driver
- Setting Browser Driver Path
- Launching WebDriver
- Basic HTML And Page DOM
- Inspect Element and Chropath Plugin
- By Locator Class
  - Static Locator Methods
- Login Script
  - Finding WebElement
- SearchContext Interface
- Quiz
- Homework

Selenium Project in Eclipse

- Create a simple Java project "**SeleniumOne**" in **Eclipse IDE**
- **Create a folder "lib":** We will use this folder to place third-party libraries
  - Right click on the project and go to "**New**" Select Folder and name it as "**lib**"
- Download **selenium-server jar**:
  - Navigate to url "**https://selenium.dev**" and click on "**Downloads**" link
  - Download latest stable version of **selenium-server-standalone-xxx.jar**
- Create "**selenium-server**" on your hard drive. Copy and paste the **selenium-server jar** there.
- Copy and paste the **selenium-server jar** in "**lib**" folder of project "**SeleniumOne**"



Include Jar in Build Path

- Create a new package in "**src**" of project and name it "**com.tests**" and create a new class "**ClassOne**" with main method in this package

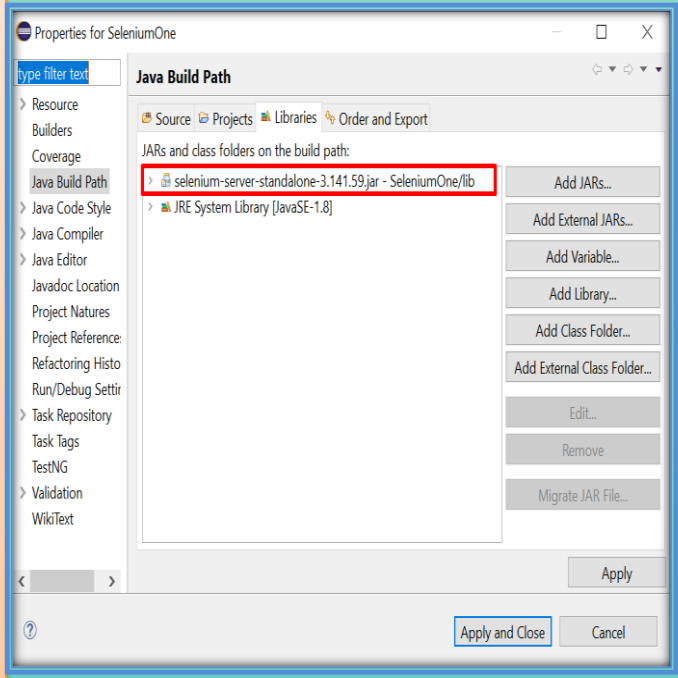
- In the main method write following statement:

  WebDriver driver = new FirefoxDriver();

  - This is a valid statement but this will **not compile** and search for **WebDriver** interface and **FirefoxDriver** class.
  - But we can not import these class files because we have not included **selenium jar** which has all **selenium classes and interfaces** in the project's **build path**.

- Steps to include **jar** in project's path:
  - Right click on project name in "**Project Explorer**"
  - Click properties, **project properties window** opens
  - Select **Java Build Path** and click "**Libraries**" tab
  - Click "**Add Jars**" button and select the **selenium jar** placed in lib of the project
  - Click "**Apply and Close**" button

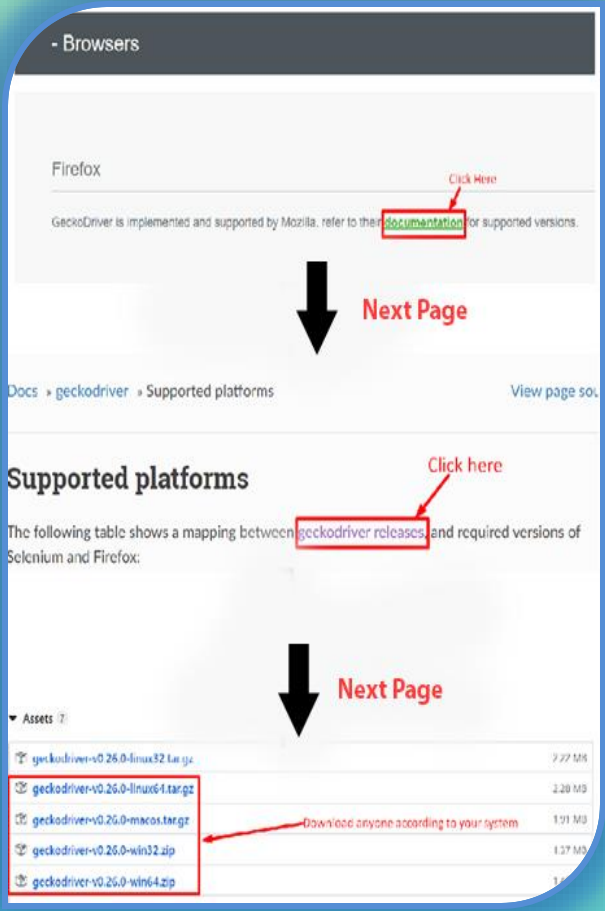
- Now when we import the **selenium classes** in "**ClassOne**" class, they can be fetched with "**import**" statement and class will compile successfully



Browser Version And Browser Driver

- Before starting with automation, you need to upgrade your **browsers** to **latest**
  - For **Chrome browser**, click on **menu button**, three identical dots stacked vertically, navigate to "**Help**" and choose "**About Google Chrome**". Latest **Chrome version** should be **79+**
  - For **Firefox browser**, click on **menu button**, three horizontal lines at the upper right side of the address bar, navigate to "**Help**" and choose "**About Firefox**". Latest **Firefox version** should be **72+**

- Download **Browser Driver** and place them in system path:
  - Navigate to url "**https://selenium.dev**" and click on "**Downloads**" link.
  - Goto "**Browsers**" section and expand it.
  - **Under Firefox**, click on "**documentation**". On New Page under Supported platforms click on "**geckodriver releases**". New page opens and on this page download **version 0.26.0** or later according to your system requirements.
  - **Under Chrome,** click on "**documentation**". On New Page click on Latest stable release "**ChromeDriver 79.0**". New page opens and on this page download **chromedriver version 79** or later according to your system requirements.



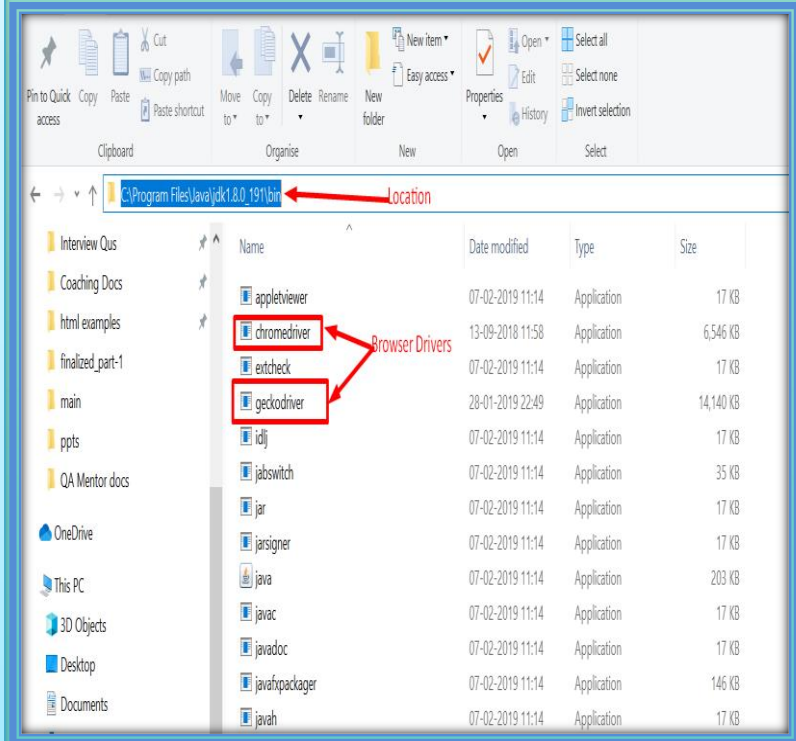
Browser Driver Path

- After downloading **Browser drivers** for **chrome** and **firefox**, place the zip file in "**selenium-server**" folder of your hard drive. After extracting the zip file we will have:
  - **geckodriver.exe**
  - **chromedriver.exe**
- Now in the selenium script or in "**ClassOne**" in main method add below statement before creating **WebDriver** instance:

  System.setProperty("webdriver.gecko.driver","D:\\selenium-server\\geckodriver.exe");
  System.setProperty("webdriver.chrome.driver","D:\\selenium-server\\chromedriiver.exe");

- JVM will add "**geckodriver**" which will talk to **Firefox browser** and "**chromedriver**" which will talk to **Chrome browser** in system path, only till script execution ends.

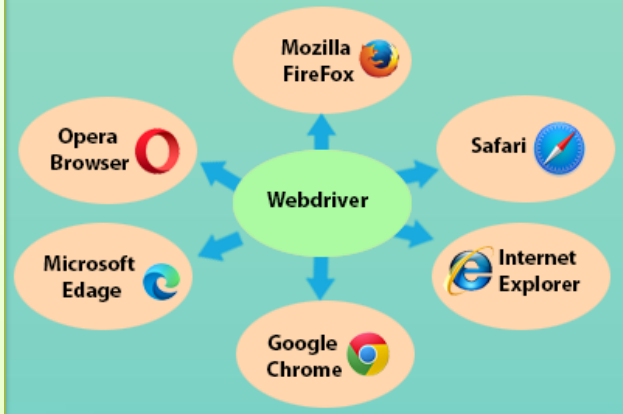
- Another way of including **Browser Drivers** is to include **Browser Drivers** in "**Path**" environment variable. As Java JDK "**bin**" is already set in "**Path**", copy **Browser Drivers** in: ***C:\Program Files\Java\jdk1.8.0_xxx\bin***
  This will set **Browser Drivers** permanently in system path.



Launching WebDriver

- **WebDriver** is the parent interface for all the **browsers**, it is capable of automating

  WebDriver driver = new FirefoxDriver();
  WebDriver driver = new ChromeDriver();
- In the first statement, **WebDriver** interface reference is instantiated with the object of **FirefoxDriver** class
- In the second statement, **WebDriver** interface reference is instantiated with the object of **ChromeDriver** class
- **FirefoxDriver** and **ChromeDriver** classes will send commands and hence JSON request to "**geckodriver**" and "**chromedriver**" respectively
- These statements will instantiate the **WebDriver** object for the session in the script and will launch the browser
- There is a method ***get(String url)*** of **WebDriver** for navigating to particular **url** of website.
  - This method uses an HTTP GET operation
  - You can not open a URL without "**http**" or "**https**" through this method. Without "**http**" or "**https**", selenium will say **Malformed URL**.




```
kage com.tests;

import org.openqa.selenium.WebDriver;

public class ClassOne {

    public static void main(String[] args) {
//      System.setProperty("webdriver.gecko.driver","D:\\selenium-server\\geckodriver.exe");
//      System.setProperty("webdriver.chrome.driver","D:\\selenium-server\\chromedriiver.exe");

        WebDriver driver = new FirefoxDriver();
        driver.get("http://app.box.com");

    }

}
```

Basic HTML And Page DOM

- Every webpage in browser is a **HTML page**.
- On **browsers**, **right click** any webpage and select "**View page source**" to view the **HTML** of page.
- **HTML** page starts with **<html>** tag and ends with **</html>** tag
- **Metadata** or behavioural properties are not displayed on page and are placed in **<head>** tag. For example **Document title**
- All components placed within **<body>** tag are displayed on page.
  The complete **HTML page** structure is known as page **DOM (Document Object Model)**
- The **DOM** represents the document as **nodes and objects**. This way, **programming languages** can connect to the **page**.
- ***<input type="text" name="FirstName" value="karan">***
  - Here "**input**" is a tagname
  - "**type**" is a attribute name and its value is "**text**"
  - Similarly "**name**" is a attribute name and its value is "**FirstName**"
- Other important HTML tags:
  - **<h1>** tag,** ** tag,**<p>** tag,**<b>** tag
  - **<input>** tag with attribute "**type**" and value "**text/submit/checkbox/radio**"
  - **<ul>** tag, **<ol>** tag, **<li>** tag, **<select>** tag
  - **<table>** tag, **<td>** tag, **<tr>** tag

```
<div style="background-color:lightblue">
  <h3>This is a heading</h3>
  <p>This is a paragraph.</p>
</div>

First name: <input type="text" name="FirstName" value="karan"><br>
<input type="submit" value="Submit">

<select>
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="mercedes">Mercedes</option>
  <option value="audi">Audi</option>
</select>

<h3> Food List:</h3>
<ul>
  <li>rice</li>
  <li>banana</li>
  <li>Milk</li>
</ul>


<a href="https://www.w3schools.com">Visit W3Schools.com!</a>
<p>My mother has <span style="color:blue">blue</span> eyes.<p>
<h3> Winners

<ol>
  <li>Raj</li>
  <li>Karan</li>
  <li>Radz</li>
</ol>
 <input type="radio" name="gender" value="male" checked> Male<br>
  <input type="radio" name="gender" value="female"> Female<br>
  <input type="radio" name="gender" value="other"> Other

<ul id="menu45" name="menubutton46" >
    <li name="Folder" class="menu-item">
        <a id="google" href="http://google.com">Google</a>
    </li>
    <li name="bookmark" class="menu-item">
        <a id="yahoo" href="http://yahoo.com">Yahoo</a>
    </li>
</ul>

</body>
</html>
```

Inspect Element and Chropath

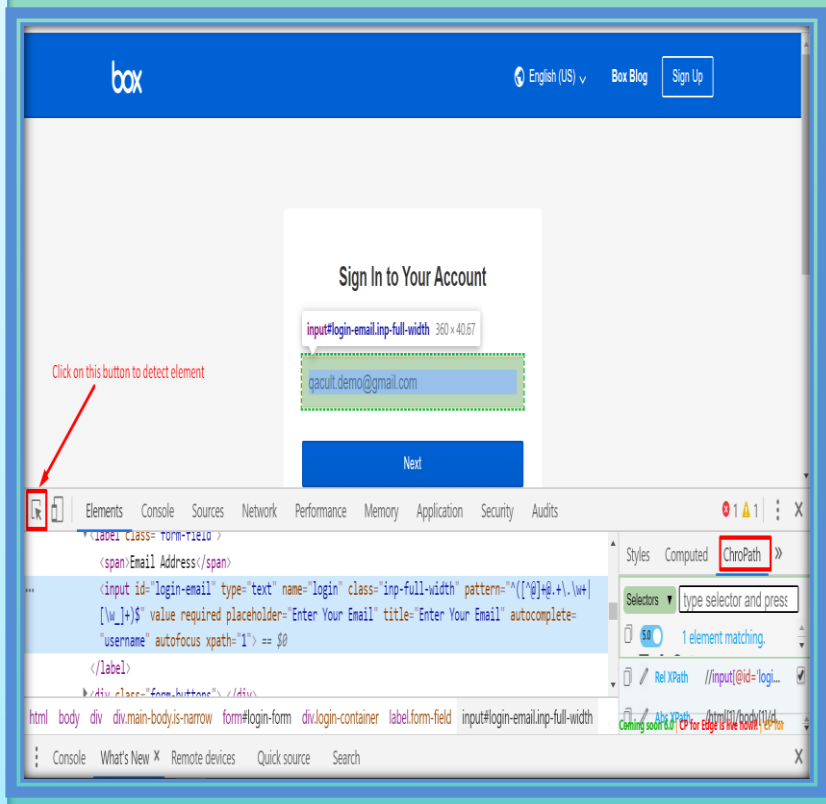
- **Inspect Element** is a powerful browser tool that lets you view and edit the **HTML** & **CSS** of web content.

- Right click on any webpage or component in **browser** and select "**Inspect Element**" or press "**Ctrl+Shift+I**" from keyboard to open **Inspect Element window** known as "**Developer tools**"

- Detect any element on webpage by clicking on button in extreme left of "**Developer tools**" window and after that click on that element

- **Chropath**: it's a **Chrome** and **Firefox** plugin to verify and generate **XPath** and **CSS selectors**

- **Chropath for firefox**:
  - **Search** for **ChroPath** in the **Find add-ons** search area and **Add to Firefox**
  - Load the web page you want to **Inspect**, Right click and select **'Inspect Element'** to launch the developer tools
  - The **ChroPath tab** will be available in your **dev tools** panel on the **extreme right** side



By Locator Class

- **By class** provides a mechanism by which to find elements within the **HTML** document.
- **By class** is known as **Locator class** because it has mainly 8 **Location Strategies** for finding element: *id*, *name*, *class*, *xpath*, *css*, *linkText*, *partialLinkText* and *tagName*
- **Location strategies linkText** and **partialLinkText** can only be used in case of **hyperlink <a>** tag
- Static Methods of **By class**:
  - **public static By id(String id)**
  - **public static By name(String name)**
  - **public static By className(String className)**
  - **public static By xpath(String xpathExpression)**
  - **public static By cssSelector(String cssSelector)**
  - **public static By linkText(String linkText)**
  - **public static By partialLinkText(String partialLinkText)**
  - **public static By tagName(String tagName)**

Please Note: All these methods return **By class** object

<input type="text" name="FirstName" id="login-name" class="textbox">

- Suppose we have a element with tagname "**input**" and attribute "**id**". Then using **By class** : ***By byObj = By.id("login-name");***
- **Location strategies name**, **id** and **class name** can be present in element as attributes directly whereas **Xpath** and **css location strategies** are developed

Finding WebElement with diffrent location strategies

Method	Syntax	Description
By ID	driver.findElement(By.id(<element ID>))	Locates an element using the ID attribute
By name	driver.findElement(By.name(<element name>))	Locates an element using the Name attribute
By class name	driver.findElement(By.className(<element class>))	Locates an element using the Class attribute
By tag name	driver.findElement(By.tagName(<htmltagname>))	Locates an element using the HTML tag
By link text	driver.findElement(By.linkText(<linktext>))	Locates a link using link text
By partial link text	driver.findElement(By.partialLinkText(<linktext>))	Locates a link using the link's partial text
By CSS	driver.findElement(By.cssSelector(<css selector>))	Locates an element using the CSS selector
By XPath	driver.findElement(By.xpath(<xpath>))	Locates an element using XPath query

Login Script

- We are going to start with "**Login Script**" on webapp "**http://app.box.com**"
  There are **3 steps** needed to perform action on any element on webpage:
  - **First locate that element with By class**
  - **Then create its WebElement**
  - **Lastly perform any action like send keys, click etc**
- In **selenium**, all components on webpage are **WebElement** though they can be **textbox**, **button**, **link**, **checkbox**, **dropdown** etc.
  HTML Component
  <input id="login-email" type="text" name="login" class="inp-full-width">
  - And its corresponding **WebElement**:
    WebElement signin =driver.findElement(By.id("login-submit"));
  - **WebElement** is a interface that represents a HTML element
  - **"driver"** is a object of **WebDriver**
  - **findElement(By by)** is a method used with **WebDriver** object which returns object of **WebElement**
  - Some Methods used to perform task on **WebElement**:
    - **void click()**
    - **void sendKeys(CharSequence... keysToSend)**

```java
import java.util.List;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;

public class FirstLogin {
    public static void main(String[] args) {
        By userFieldBy = By.id("login-email");
        WebDriver driver = new FirefoxDriver();
        driver.get("https://app.box.com");
        WebElement userField = driver.findElement(userFieldBy);
        userField.sendKeys("nancydhingra131@gmail.com");
        WebElement nxtBtn = driver.findElement(By.id("login-submit"));
        nxtBtn.click();
        List<WebElement> passList = driver.findElements(By.id("password-login"));
        System.out.println(passList.size());
        passList.get(0).sendKeys("nancy131");
        WebElement loginBtn = driver.findElement(By.id("login-submit-password"));
        loginBtn.click();
```

SearchContext Interface

- **SearchContext interface** is the parent interface of **WebDriver** interface and is at root of hierarchy.

- It has only 2 methods:
  - **WebElement findElement(By by)** – This method takes **By class** object as argument and returns **WebElement** object
  - **List<WebElement> findElements(By by)** - This method also takes **By class** object as argument and returns java.util.List of **WebElement**
  - Both the methods are used with **WebDriver object** as they are inherited by the WebDriver and hence respective Browsers.

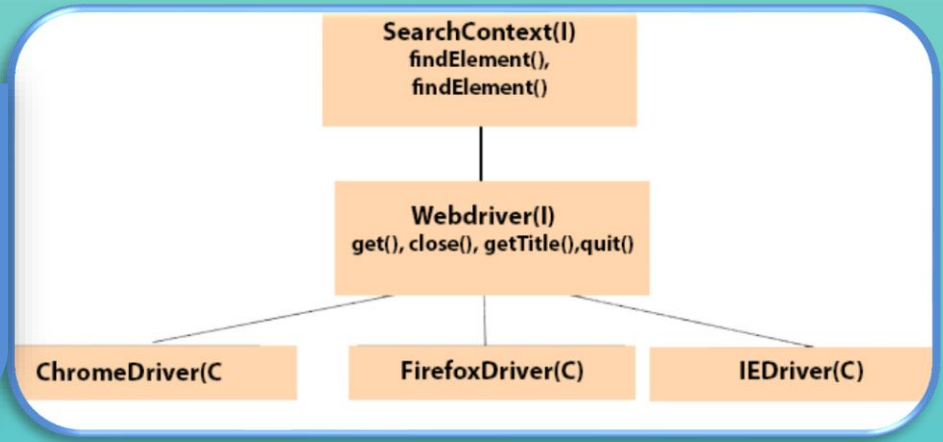
It makes use of java.util.List

```
List<WebElement> passList = driver.findElements(By.id("password-login"));

System.out.println(passList.size());

passList.get(0).sendKeys("nancy131");
```

Get the WebElement first to perform action



Quiz

- Which jar will you require to work on **Selenium project** and how will you include the jar in **project's build path**?
- How will you include **BrowserDriver's** path in your script and how can you set their path **permanently**?
- What is **page DOM**?
- Where is page title found in **HTML** document?
- How will you find element's corresponding **HTML** on a webpage?
- How is **WebDriver API** related to **BrowserDrivers**?
- What is the use of **By class**, tell its **location strategies** methods and what do these method return?
- What is the difference between **findElement()** and **findElements()** methods?
- Where is **SearchContext interface** located in **WebDriver** hierarchy?

Homework

- In a WebDriver script do:
  - Navigate to http://app.box.com
  - Click on "**Signup**" button and then "**Contact**" link to navigate on "**Contact Us**" page
  - On "**Contact Us**" page enter value for **first name**, **last name**, **business email**, **Phone**, **Job title** and **company name**
  - Select **Company size** and **Country** value from the dropdown
  - Fill some value in **TextArea** "**How can sales help you?"**
  - Click on "**Contact Us**"
  - Verify "**Thank You**" message